Terraform-03: Command-Line Variable Overrides

Objective

To practise overriding default Terraform input variables using command-line flags and verify the generated local file.

Steps Performed

Step 1: Review Terraform Configuration

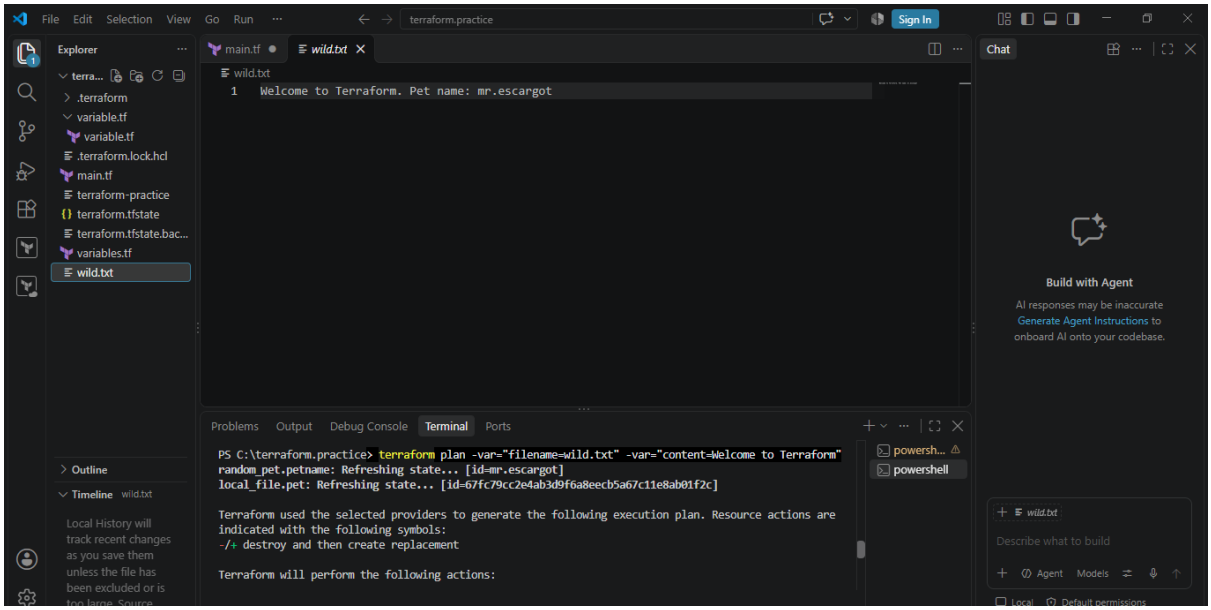
Reviewed the existing `main.tf` file containing the `random_pet` and `local_file` resources.

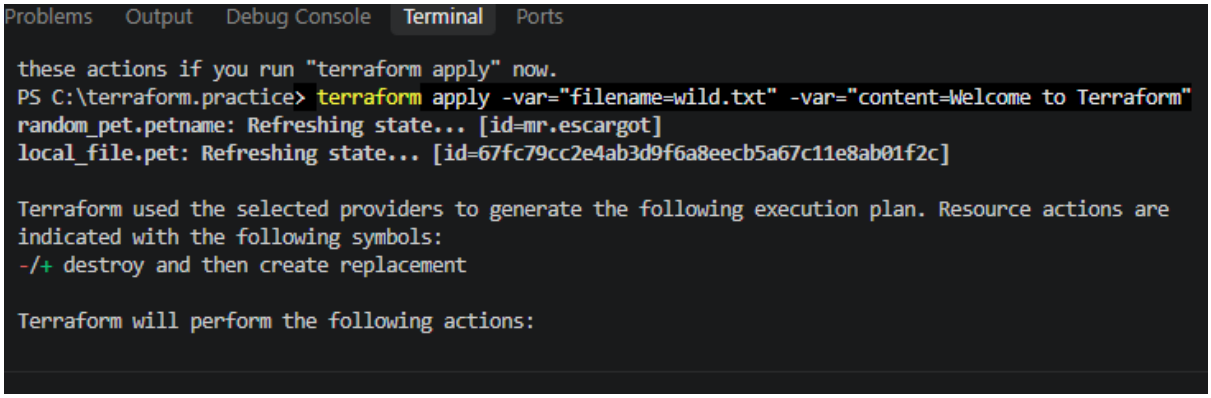
- `random_pet` generates a random pet name.
- `local_file` creates a file using the specified filename and content.
- The generated pet name is included in the file content using a resource attribute reference.
- execute Terraform Plan

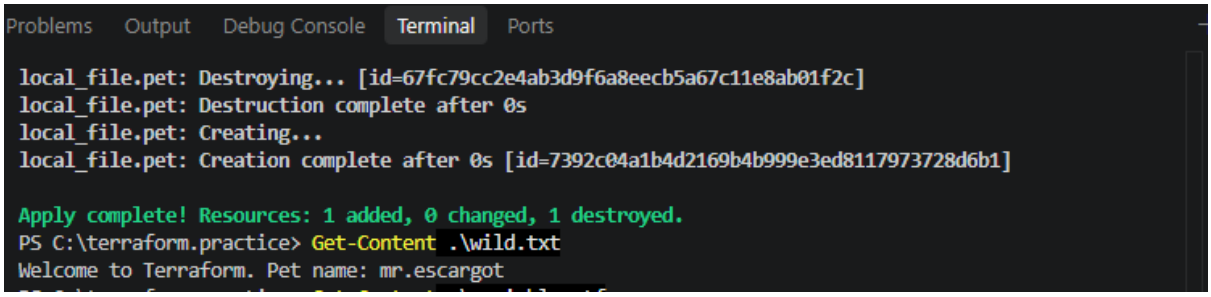
  terraform plan -var="filename=wild.txt" -var="content=Welcome to Terraform"

  terraform apply -var="filename=wild.txt" -var="content=Welcome to Terraform"

  Get-Content .\wild.txt







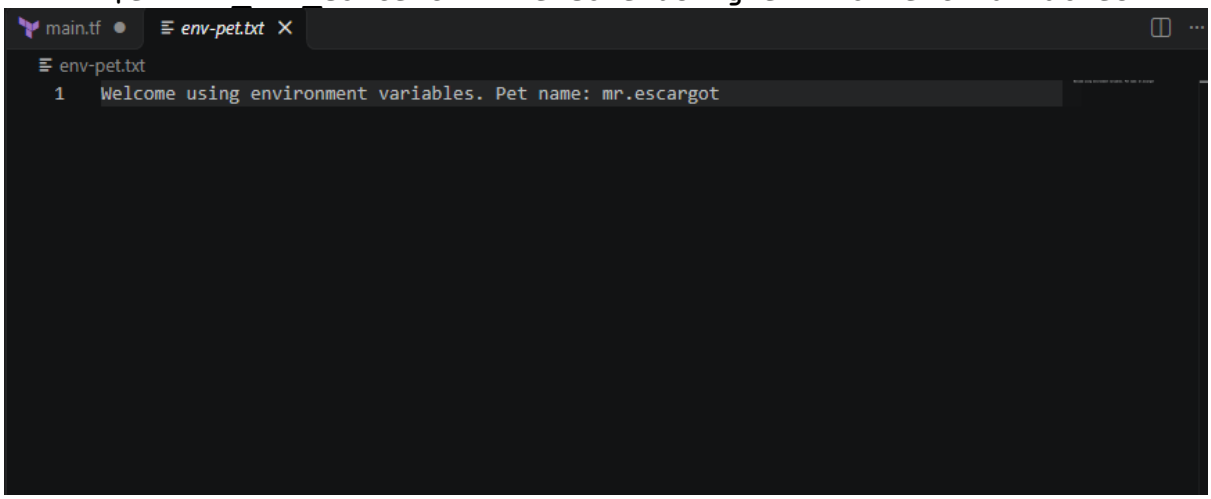
Terraform Variable Management Using Environment Variables

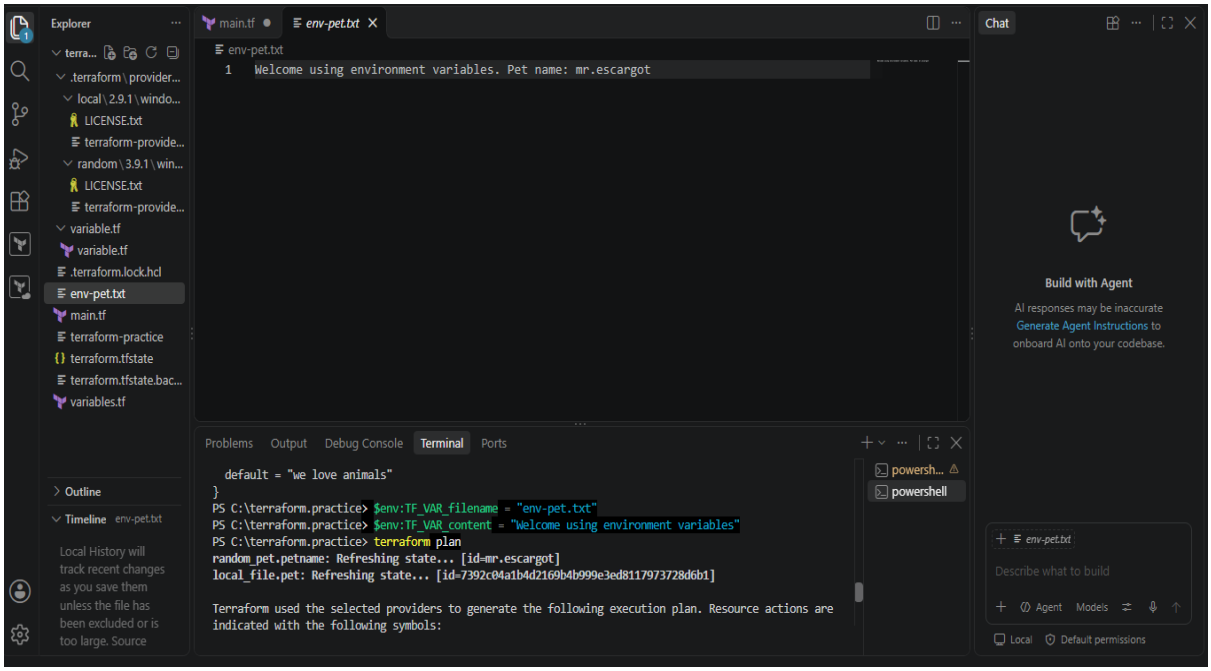
Objective: To pass Terraform input variable values through environment variables and verify the generated file.

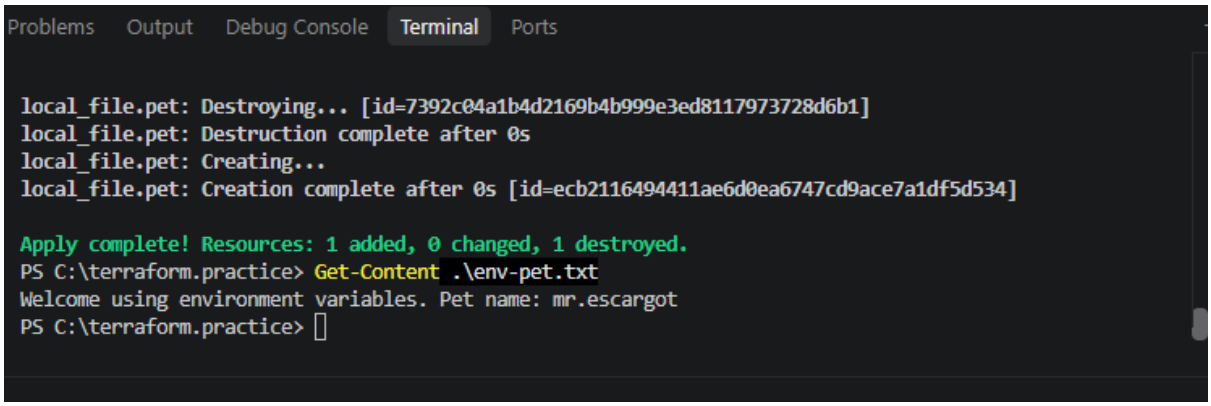
Steps performed:

1. Defined `TF_VAR_filename` and `TF_VAR_content` in PowerShell.
2. Executed `terraform plan` to review the proposed changes.
3. Executed `terraform apply` to create the file.
4. Verified the generated file using `Get-Content`.

```
$env:TF_VAR_filename = "env-pet.txt"
 $env:TF_VAR_content = "Welcome using environment variables"
```







Variable Definition Using :- `terraform.tfvars`

1. Objective

To learn how to define Terraform input variables using the automatically loaded `terraform.tfvars` file and override default variable values.

2. Steps Performed

Step 1: Created the variable definition file

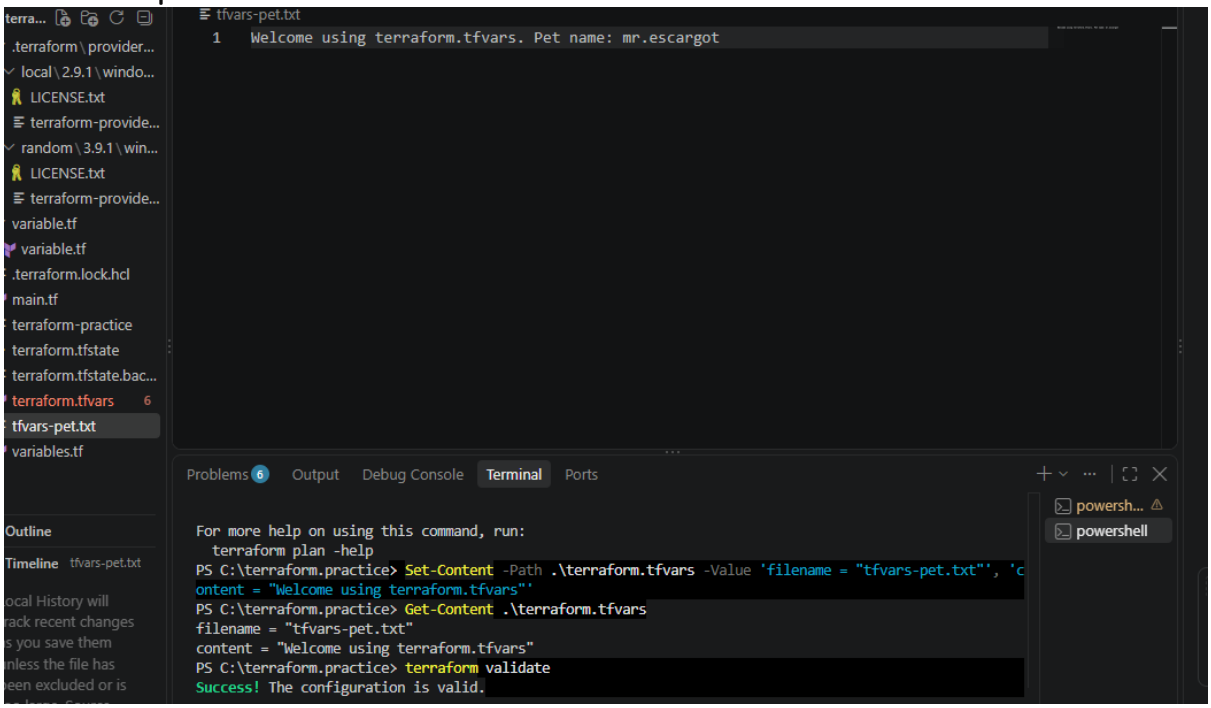
Created `terraform.tfvars` in the existing `C:\terraform.practice` project directory.

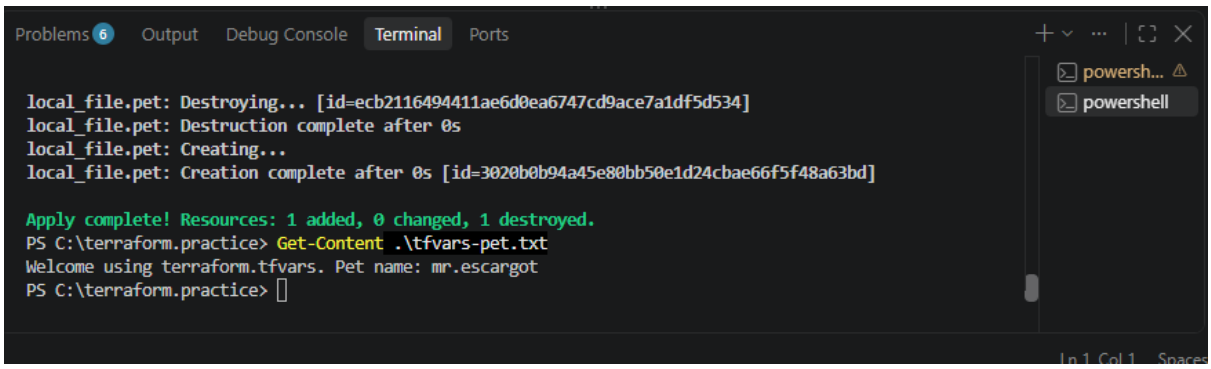
Configured the following variables:

```
filename = "tfvars-pet.txt"
content  = "Welcome using terraform.tfvars"
```

Executed:

Terraform validate
terraform plan





TERRAFORM AUTOMATIC VARIABLE DEFINITION USING .AUTO.TFVARS

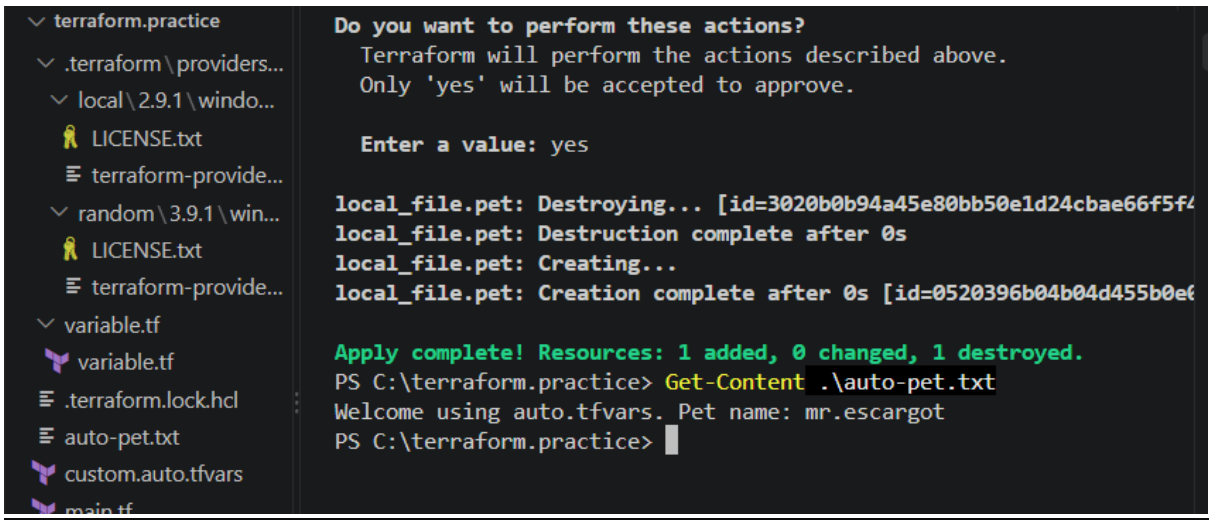
Tasks performed:

1. Created `custom.auto.tfvars`.
2. Defined custom filename and content values.
3. Executed `terraform plan` to verify variable precedence.
4. Applied the configuration using `terraform apply`.
5. Verified the generated `auto-pet.txt` file.

```
PS C:\terraform.practice> Set-Content -Path .\custom.auto.tfvars -Valu
e 'filename = "auto-pet.txt"', 'content = "Welcome using auto.tfvars"'
PS C:\terraform.practice>
PS C:\terraform.practice> Set-Content -Path .\custom.auto.tfvars -Valu
e 'filename = "auto-pet.txt"', 'content = "Welcome using auto.tfvars"'

PS C:\terraform.practice> Get-Content .\custom.auto.tfvars
filename = "auto-pet.txt"
content = "Welcome using auto.tfvars"
random_pet.petname: Refreshing state... [id=mr.escargot]
local_file.pet: Refreshing state... [id=3020b0b94a45e80bb50e1d24cbae66f5f48a63bd]

Terraform used the selected providers to generate the following execution plan. Resource actions
indicated with the following symbols:
-/+ destroy and then create replacement
```

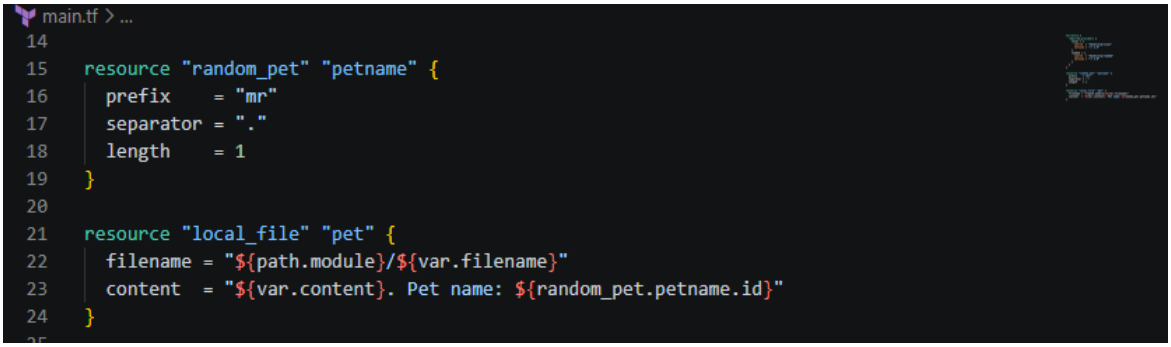


Terraform Custom Variable File

Objective: Pass Terraform variables using a custom `.tfvars` file.

Steps completed:

1. Created `dev.tfvars` containing the filename and content variables.
2. Executed `terraform plan -var-file="dev.tfvars"`.
3. Applied the configuration using `terraform apply -var-file="dev.tfvars"`.
4. Verified that `dev-pet.txt` was created with the expected content.



```
PS C:\terraform.practice> terraform plan -var-file="dev.tfvars"
random_pet.petname: Refreshing state... [id=mr.escargot]
local_file.pet: Refreshing state... [id=0520396b04b04d455b0e0a3f1108c1b2098dbfc8]

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

  # local_file.pet must be replaced
-/+ resource "local_file" "pet" {
      ~ content              = "Welcome using auto.tfvars. Pet name: mr.escargot" -> "Welcome us
ing a custom variable file. Pet name: mr.escargot" # forces replacement
      ~ content_base64sha256 = "WhSLhXOwUsU81LunacNrrTcMqO3hoRndHaamQUnuvXw=" -> (known after ap
ply)
      ~ content_base64sha512 = "bRtU8zORAABjaIxvqP8m3IsNwgYh9GKt7UjT6z/voVU494mlgpzQZa1Wk3PzeK+d
wMUV8DJwXebbcIiEs0pziw==" -> (known after apply)
      ~ content_md5          = "e188f083e49425669ad46e83cb344e06" -> (known after apply)
      ~ content_sha1         = "0520396b04b04d455b0e0a3f1108c1b2098dbfc8" -> (known after apply)
      ~ content_sha256       = "5a148b8573b052c53cd4bba769c36bad370ca8ede1a119dd1da6a64149eebd7c
" -> (known after apply)
      ~ content_sha512       = "6d1b54f33391000063688c6fa8ff26dc8b0dc20621f462aded48d3eb3fefa155
38f789a5829cd065ad569373f378af9dc0c515f032705de6db708884b34a738b" -> (known after apply)
      ~ filename             = "./auto-pet.txt" -> "./dev-pet.txt" # forces replacement
      ~ id                   = "0520396b04b04d455b0e0a3f1108c1b2098dbfc8" -> (known after apply)
        # (2 unchanged attributes hidden)

Plan: 1 to add, 0 to change, 1 to destroy.
```

```
LICENSE.txt
terraform-provide...
variable.tf
variable.tf
.terraform.lock.hcl
custom
custom.auto.tfvars
dev-pet.txt
dev.tfvars
main.tf
terraform-practice
terraform.tfstate
terraform.tfstate.bac...

local_file.pet: Destroying... [id=0520396b04b04d455b0e0a3f1108c1b2098dbfc8]
local_file.pet: Destruction complete after 0s
local_file.pet: Creating...
local_file.pet: Creation complete after 0s [id=675dd6f0c2be28048e516ff3e547f69aa403b67c]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.
PS C:\terraform.practice> Get-Content .\dev-pet.txt
Welcome using a custom variable file. Pet name: mr.escargot
PS C:\terraform.practice>
```

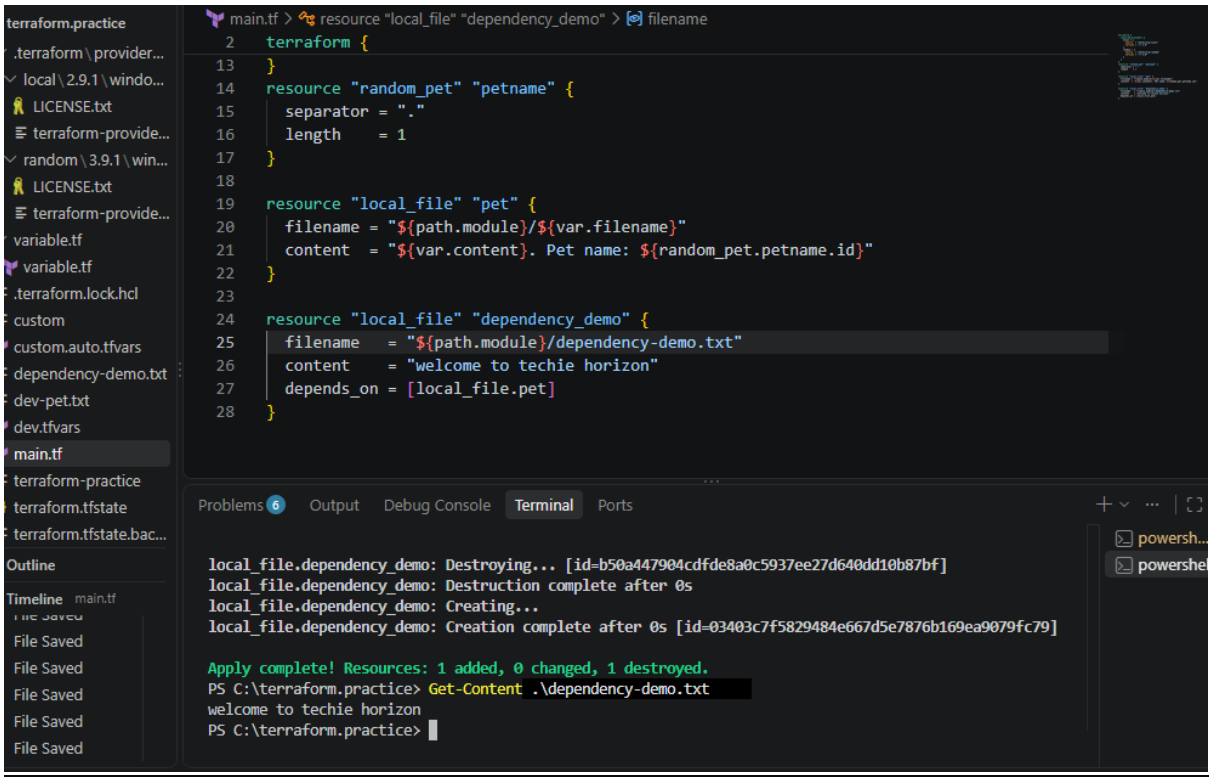
Implicit and Explicit Dependencies

Objective: Understand how Terraform determines the order in which resources are created.

Configuration used: `main.tf`

Points completed

1. Implicit dependency: The `local_file.pet` resource references `random_pet.petname.id`. Terraform automatically creates the pet name before creating the file.
2. Explicit dependency: Added `depends_on = [local_file.pet]` to the `local_file.dependency_demo` resource.
3. Validation: Executed `terraform fmt`, `terraform validate`, and `terraform plan`.
4. Apply: Applied the configuration using `terraform apply -var-file="dev.tfvars"`.
5. Verification: Confirmed that `dependency-demo.txt` was created with the expected content.

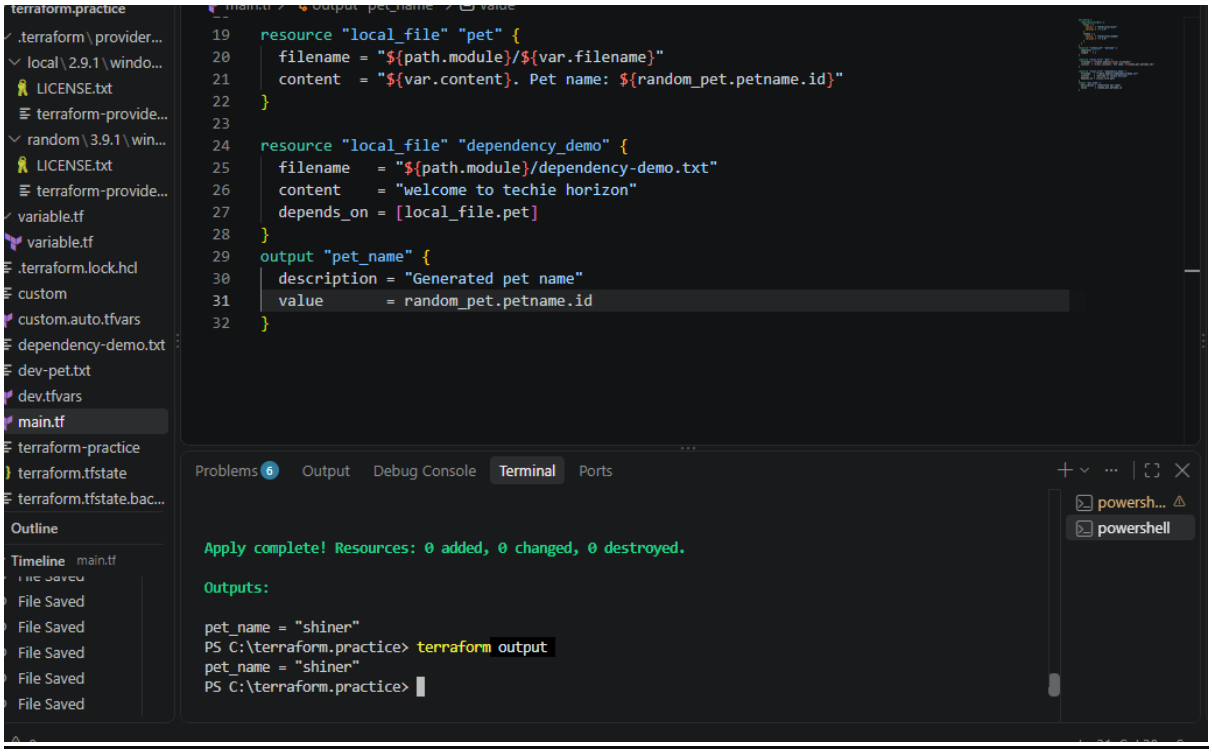


Documentation: Output Values

Objective: Display useful information generated by Terraform resources.

Points completed

1. Created an output variable: Added an `output "pet_name"` block in `main.tf`.
2. Resource attribute reference: Used `random_pet.petname.id` to retrieve the generated pet name.
3. Validation: Executed `terraform fmt` and `terraform validate`.
4. Plan and apply: Ran `terraform plan` and `terraform apply` using `dev.tfvars`.
5. Display outputs: Used `terraform output` to display all output values.
6. Specific output: Used `terraform output pet_name` to display only the generated pet name.



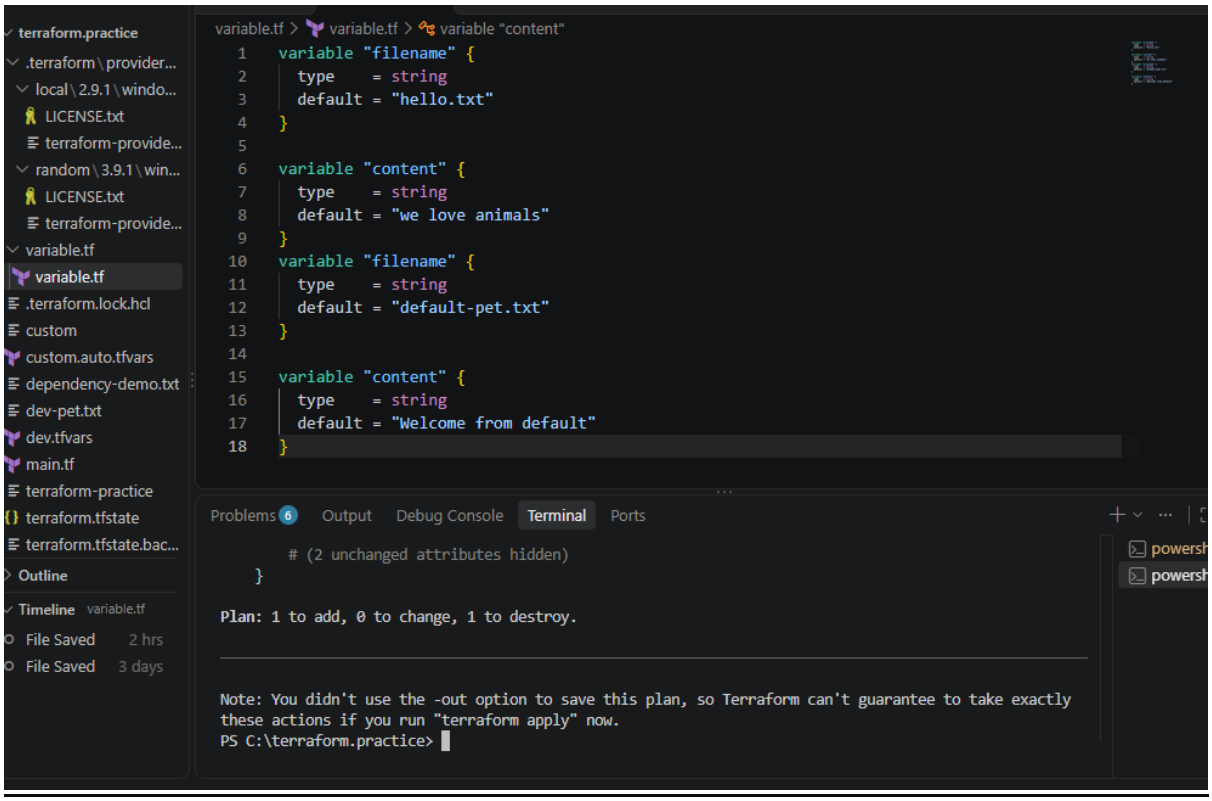
Variable Precedence

Objective: Understand how Terraform selects variable values when the same variable is defined in multiple locations.

Points completed:

1. Defined default variable values in `variables.tf`.
2. Set variable values using `terraform.tfvars`.
3. Passed variables through environment variables using the `TF_VAR_` prefix.
4. Overrode variable values using the command-line `-var` flag.

5. Compared Terraform plans to observe which variable value takes precedence.
6. Removed the temporary environment variable after testing.

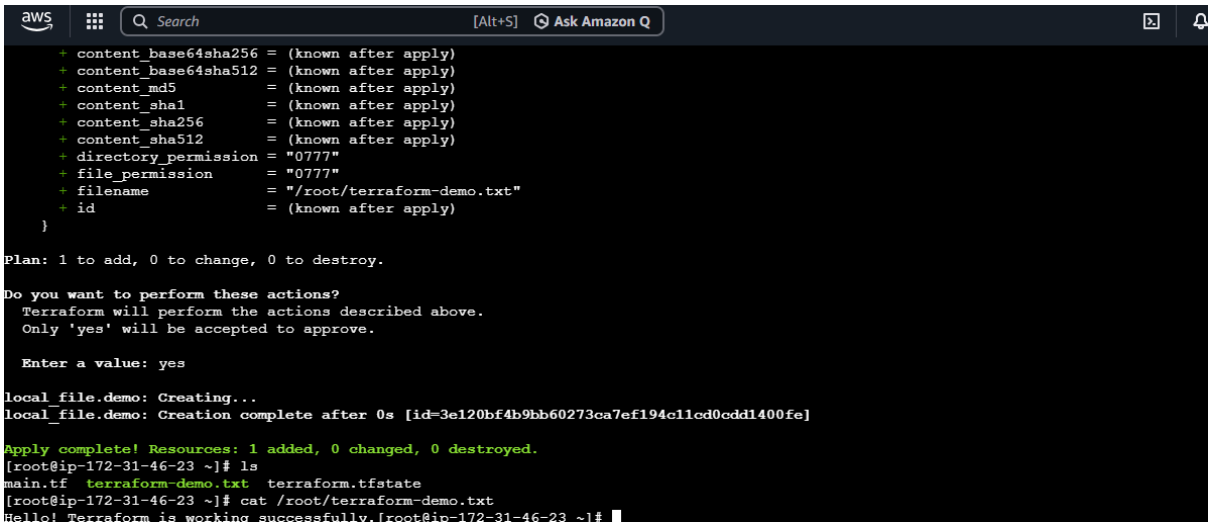


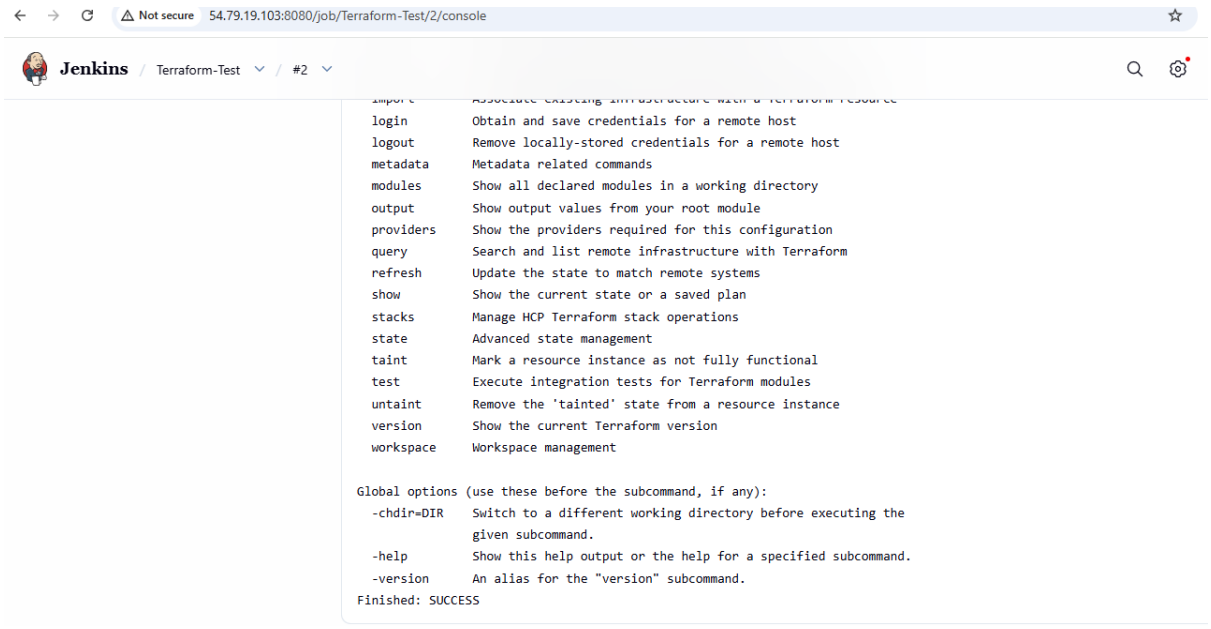
Terraform Integration with Jenkins

Objective: Integrate Terraform with Jenkins to execute Infrastructure as Code commands through automated jobs.

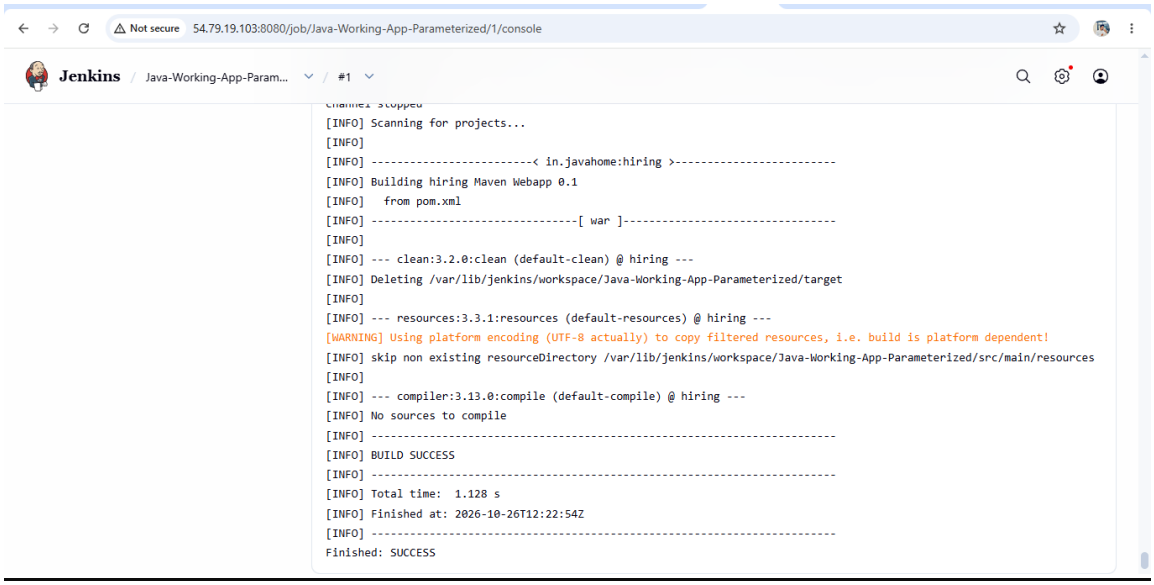
1. Installed the Terraform plugin through Jenkins Plugin Manager.
2. Installed Terraform on the EC2 host or Jenkins build agent.

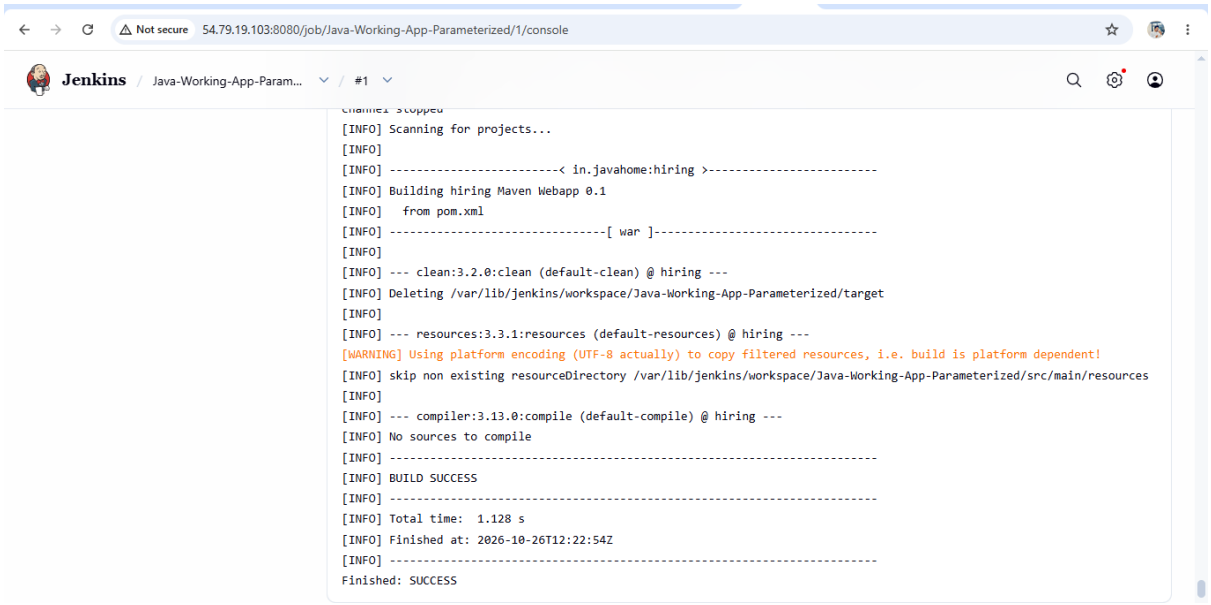
3. Configured Terraform in Jenkins tool settings, where supported.
4. Created a test Freestyle job to verify Terraform access.
5. Verified Terraform execution through Jenkins Console Output.
6. Prepared Jenkins to validate and plan Terraform configurations.

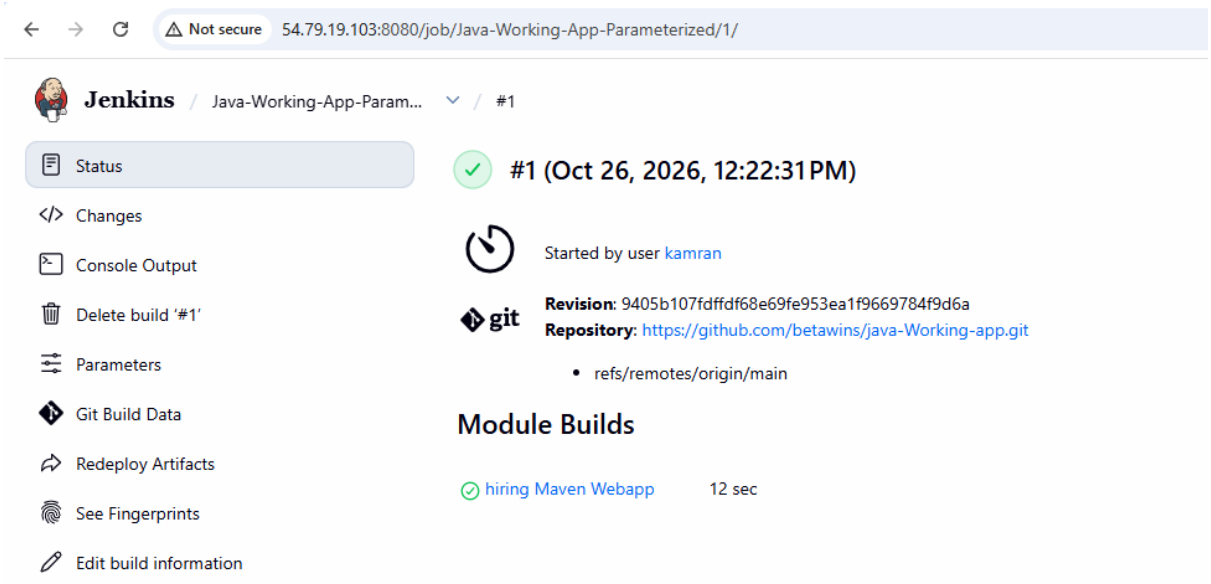


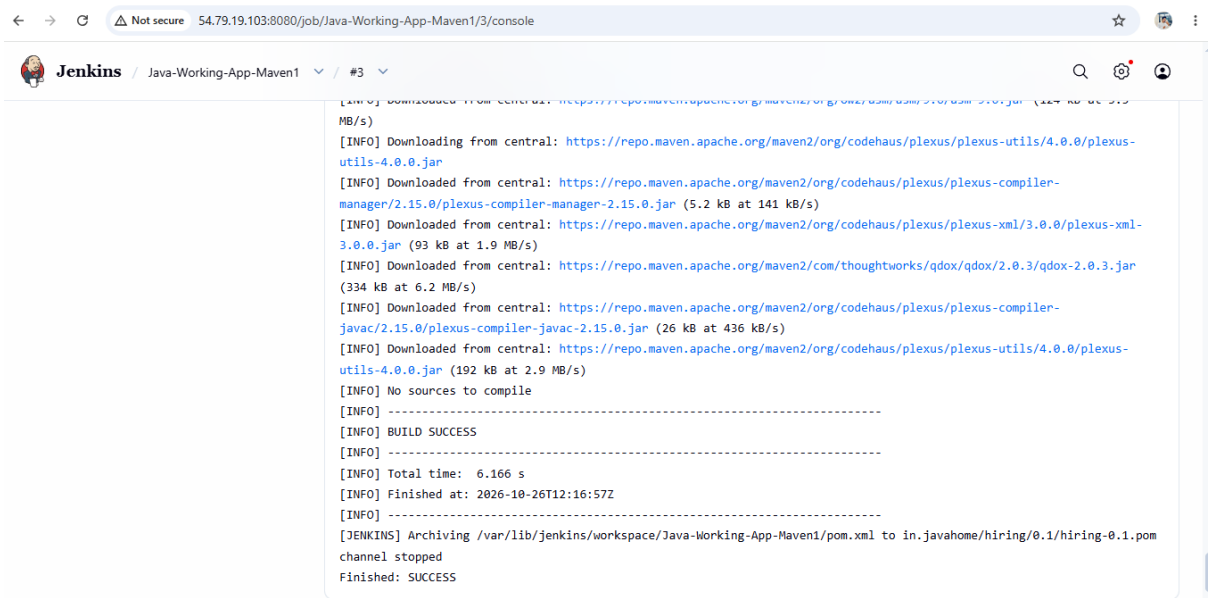


- Created a Maven Project job to clone and compile a Java application.
- Configured Git SCM using the supplied GitHub repository.
- Configured Maven goals `clean compile`.
- Created a parameterized Maven job with a `git` parameter.
- Executed both jobs and checked their console output.
- Studied Jenkins global environment variables and their uses.






Not secure
54.79.19.103:8080/job/Java-Working-App-Parameterized/1/
Jenkins
Java-Working-App-Param...
#1
Status
Changes
Console Output
Delete build '#1'
Parameters
Git Build Data
Redeploy Artifacts
See Fingerprints
Edit build information
#1 (Oct 26, 2026, 12:22:31 PM)
Started by user kamran
git
Revision: 9405b107fdffdf68e69fe953ea1f9669784f9d6a
Repository: https://github.com/betawins/java-Working-app.git
refs/remotes/origin/main
Module Builds
hiring Maven Webapp
12 sec


Not secure
54.79.19.103:8080/job/Java-Working-App-Maven1/3/console
Jenkins
Java-Working-App-Maven1
#3
MB/s)
[INFO] Downloading from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/4.0.0/plexus-utils-4.0.0.jar
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-manager/2.15.0/plexus-compiler-manager-2.15.0.jar (5.2 kB at 141 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-xml/3.0.0/plexus-xml-3.0.0.jar (93 kB at 1.9 MB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/com/thoughtworks/qdox/qdox/2.0.3/qdox-2.0.3.jar (334 kB at 6.2 MB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-compiler-javac/2.15.0/plexus-compiler-javac-2.15.0.jar (26 kB at 436 kB/s)
[INFO] Downloaded from central: https://repo.maven.apache.org/maven2/org/codehaus/plexus/plexus-utils/4.0.0/plexus-utils-4.0.0.jar (192 kB at 2.9 MB/s)
[INFO] No sources to compile
[INFO] --
[INFO] BUILD SUCCESS
[INFO] --
[INFO] Total time: 6.166 s
[INFO] Finished at: 2026-10-26T12:16:57Z
[INFO] --
[JENKINS] Archiving /var/lib/jenkins/workspace/Java-Working-App-Maven1/pom.xml to in.javahome/hiring/0.1/hiring-0.1.pom
channel stopped
Finished: SUCCESS

What are global variables in Jenkins?

Jenkins exposes environment variables containing information about the current job, build, agent, workspace, and other details. These are useful in build scripts and pipelines.



GitHub

+1

Common Jenkins environment variables:

Variable	Purpose
`BUILD_NUMBER`	Current build number
`BUILD_ID`	Unique build identifier
`BUILD_URL`	URL of the build
`JOB_NAME`	Name of the Jenkins job
`JOB_BASE_NAME`	Job name without folder path
`WORKSPACE`	Directory where the job runs
`JENKINS_URL`	Jenkins server URL, if configured
`NODE_NAME`	Name of the node running the build
`EXECUTOR_NUMBER`	Executor slot number
`BUILD_TAG`	Unique build tag
`JAVA_HOME`	Configured Java installation path, when available
`GIT_URL`	Git repository URL, when provided by Git integration
`GIT_COMMIT`	Checked-out Git commit, when provided by Git integration